

Where Is the Bee? Detecting Tiny Pollinators with a Single Collaborative-Head Transformer

Junsu Kim Seungryul Baek
UNIST, South Korea

Abstract

The CVPPA@ECCV 2026 BuzzSpot Challenge asks us to detect bees, bumblebees, hoverflies, and moths in 1920×1080 field keyframes. Its annotations carry 2 difficulties: the median box occupies 0.16% of a frame, and bees account for 80% of the labels. To cope with the small boxes, we compare 10 recorded detector configurations on held-out keyframes; plain Co-DINO with a Swin-L backbone has the highest mAP in this comparison, so we select it. Training then addresses the bee dominance in 2 ways: fine-tuning on a crop-mosaic pool in which the combined annotation share of the 3 rare classes rises from 19.9% to 55.1%, and a class-weighted simplex equiangular tight frame (ETF) loss that pulls the projected states of matched decoder queries toward fixed class directions. The full schedule spans 12+3+2 epochs. Without inference-time ensembling or test-time augmentation, we rank first on FinalTest at 0.5062 mAP@[.5:.95]. Code: github.com/jjunsss/BeeHunters.

1. Introduction

The CVPPA@ECCV 2026 BuzzSpot Challenge asks us to localize bees, bumblebees, hoverflies, and moths in 1920×1080 field keyframes [1]. Its annotations make this hard in 2 ways: the median labeled box covers 0.16% of a frame, the area of a 58×58 -pixel square, leaving few pixels to localize an insect and identify its class; and bees contribute 9,611 of the 12,000 boxes, so most of the training signal comes from one class.

For the small boxes, Fig. 2 compares plain Co-DINO against 9 YOLO and RTMDet configurations on held-out keyframes. Their training and inference protocols differ, so we use the comparison only to select the detector family; Co-DINO with Swin-L has the highest recorded mAP, and we use it in the final system [2, 3].

With the detector fixed, the bee dominance remains. We treat class exposure as a design variable [4]: object-centered crops of bumblebees, hoverflies, or moths receive higher sampling priority, and Mosaic augmentation [5] tiles 4 of these sampled views into one training image, so the detector

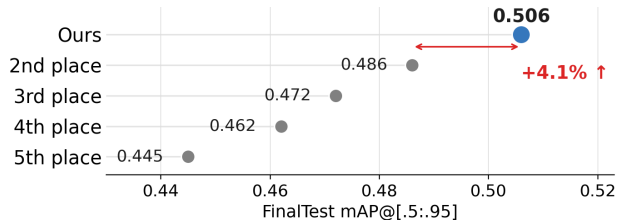


Figure 1. Official FinalTest leaderboard.

sees rare-class objects more often without new labels. For feature geometry, we transfer fixed-prototype alignment [6–8] to a class-weighted simplex equiangular tight frame (ETF) loss, which supervises each projected, Hungarian-matched decoder state toward a fixed direction for its class.

We organize these choices in a 3-stage schedule: 12 epochs adapting the detector on all labeled keyframes, 3 with class-aware crop Mosaic to shift class exposure, and 2 back on the original keyframes at a low learning rate, with the ETF loss active throughout. Without inference-time ensembling or test-time augmentation, a choice made for practical use, our checkpoint tops the official FinalTest leaderboard¹ in Fig. 1 at 0.5062 mAP@[.5:.95].

2. Method

We select the detector on held-out mAP (Sec. 2.1), tailor its matched-query supervision to the annotation distribution (Sec. 2.2), and control its data exposure through a 3-stage training schedule (Sec. 2.3).

2.1. Co-DINO with a Swin-L backbone

On the 932 held-out keyframes of Fig. 2, plain Co-DINO with Swin-L reaches 0.562 mAP against 0.465 for the best of 9 YOLO and RTMDet configurations and corrects the shown miss and class confusion, so we select it [2, 3].

We keep Swin-L because its hierarchy retains a high-resolution output and multi-scale features. Swin-L produces 4 stage outputs, which the neck maps to 5 detector levels with 256 channels each. We initialize from an Objects365-to-COCO checkpoint [9, 10].

¹<https://www.codabench.org/competitions/16441/>

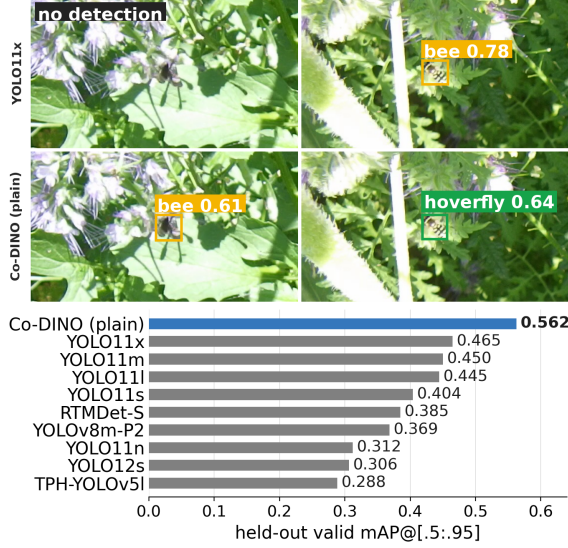


Figure 2. **Detector-family comparison.** Top: YOLO11x misses a bee and confuses a hoverfly; yet, plain Co-DINO detects both at score ≥ 0.5 . Bottom: recorded held-out mAP@[.5:.95].

Co-DINO extends DINO [11] with 2 auxiliary heads that run only during training: an RoI head and an ATSS head, each assigning many positives per object on the shared encoder features, which densifies supervision and forms extra training queries. The third category head is DINO’s query head, which matches one query to one object through Hungarian assignment and alone serves inference; the auxiliary heads strengthen the shared features that the query head uses. All 3 category heads predict the same 4 classes, so the bee-dominated labels affect each training path.

2.2. Fixed-ETF query regularization

We apply fixed-ETF regularization only to the Co-DINO query head’s Hungarian-matched final-layer queries. We retain its classifier and define 4 fixed, unit-norm simplex prototypes $\{\mathbf{m}_c\}_{c=1}^4 \subset \mathbb{R}^{256}$ with equal pairwise angles [6–8, 12]. These equally separate the class targets; class weights change only their loss contribution.

Following Fig. 3, we exclude denoising queries, match final-layer predictions to ground truth, and use positive matches to select decoder states. We apply a learned bias-free linear projection, ℓ_2 -normalize its output as $\hat{\mathbf{p}}_i$, and compare it with the matched-class prototype $\mathbf{m}_{y_i}$. The class-weighted loss over matched set $\mathcal{P}$ is

$$\mathcal{L}_{\text{ETF}} = \frac{1}{\sum_{i \in \mathcal{P}} w_{y_i}} \sum_{i \in \mathcal{P}} w_{y_i} \frac{1}{2} (\langle \hat{\mathbf{p}}_i, \mathbf{m}_{y_i} \rangle - 1)^2, \quad (1)$$

where y_i is the class of match i and w_{y_i} is its class weight. The class weights are $w_c = 1.0, 1.5, 4.0$, and 2.5 for bee, bumblebee, hoverfly, and moth, weighting every rare class above bee. The total training loss then adds $0.1 \mathcal{L}_{\text{ETF}}$ to Co-DINO’s unchanged detection terms.

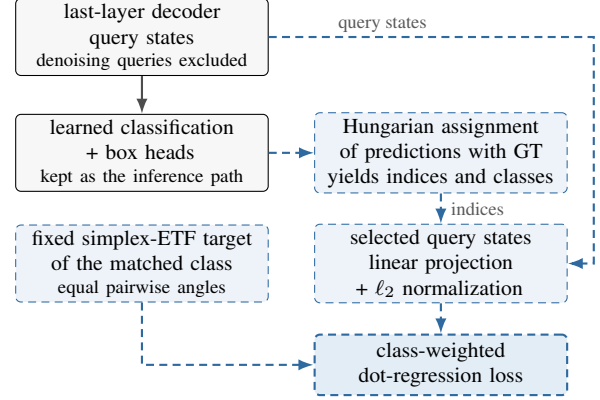


Figure 3. **Fixed-ETF branch.** We exclude denoising queries before matching the final-layer predictions to ground truth. Matched indices select decoder states, matched classes select fixed targets, and dashed lines mark the training-only path.

2.3. 3-stage final training

The loss above sets the supervision; what remains is the data each stage presents, which Fig. 4 organizes into 3 stages. We first adapt the pretrained detector to the BuzzSpot classes and full-frame context by training on all 6,207 train+valid keyframes for 12 epochs at learning rate 10^{-4} .

For stage 2, we train for 3 epochs at 10^{-5} on a class-aware crop-mosaic pool. We build this pool by sampling 5,993 object-centered views from the original training split, with base class multipliers $[0.75, 10, 3, 10]$ for bee, bumblebee, hoverfly, and moth and square crop side lengths of 192–384 pixels. Heuristic factors then adjust each annotation’s sampling weight: it doubles when the box area falls below that of a 32×32 -pixel square, multiplies by 1.5 for blur, and by another 1.5 for occlusion level 2 or higher. The crop center jitters by up to 15% of its side length along each axis, and every source box whose intersection retains at least 35% of its original area is remapped with its category and attributes preserved. All 932 uncropped validation images also join the pool to retain full-frame samples. Tab. 1 reports its composition before Mosaic: the 3 rare classes account for 55.1% of the annotations, compared with 19.9% in the original data. Fig. 5 shows 4 views on a 2048×2048 Mosaic canvas with a 1024×1024 base scale [5].

Mosaic changes the full-frame context and scale distribution, so we finish with 2 epochs on the original train+valid keyframes at a lower learning rate of 5×10^{-6} .

3. Experiments

3.1. Dataset and protocol

The train+valid annotations used for final training contain 6,207 keyframes and 12,000 boxes across the 4 BuzzSpot classes; FinalTest contains 4,763 keyframes with hidden annotations. For scale analysis, we use the COCO API’s small-object range and count boxes whose area is at most that

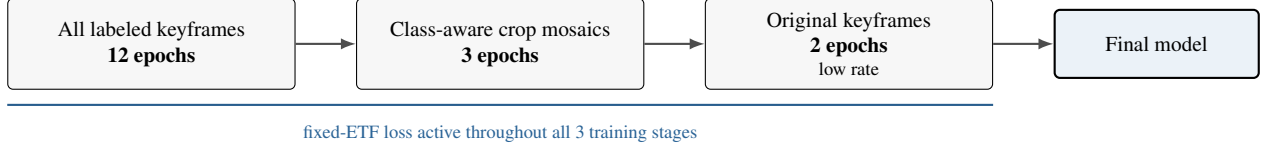


Figure 4. **Training schedule.** We train for 12 epochs on the original keyframes, 3 with class-aware crop Mosaic, and 2 on the original keyframes at a lower learning rate.

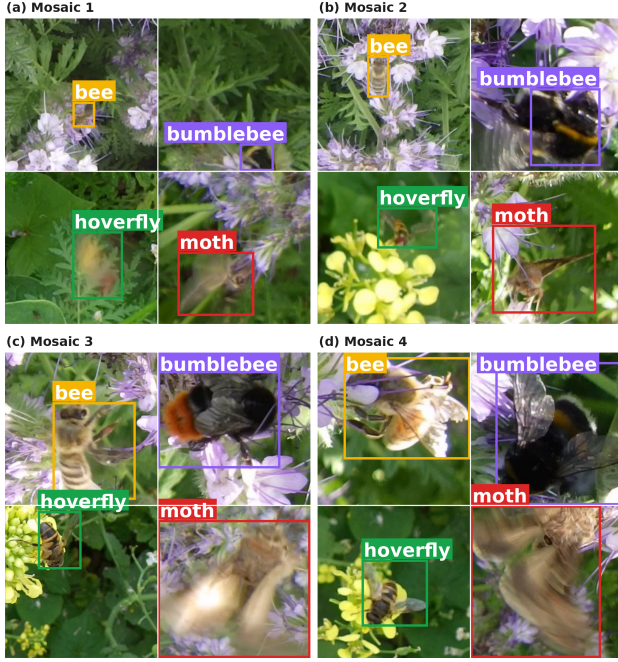


Figure 5. **Visualization of mosaic augmentation.**

of a 32×32 -pixel square as small [13]. This includes 1,018 of 1,848 hoverfly boxes (55.1%), compared with 425 of 9,611 bee boxes (4.4%), 1 of 267 bumblebee boxes (0.4%), and 0 of 274 moth boxes. For detector selection, we train each model on the training videos and evaluate it on the 932 validation keyframes. We average COCO-style mAP over IoU thresholds from 0.50 to 0.95 in 0.05 steps. We then train the final model on all train+valid keyframes. With the validation split now in-sample, FinalTest remains the only held-out evaluation.

3.2. Implementation details

Our released recipe trains data-parallel with PyTorch [14] on 4 NVIDIA A100 80 GB GPUs at batch size 1 per GPU. We optimize with AdamW, use weight decay 10^{-4} , clip the gradient norm at 0.1, and set the backbone learning rate to 0.1 times each stage’s rate from Sec. 2.3. Every stage fol-

Training source	Bee	B.bee	Hover.	Moth	Rare share
Original train+valid	9,611	267	1,848	274	19.9%
Crop-mosaic pool	3,420	799	2,628	777	55.1%

Table 1. **Class composition.** Counts precede Mosaic augmentation; rare share combines bumblebee, hoverfly, and moth.

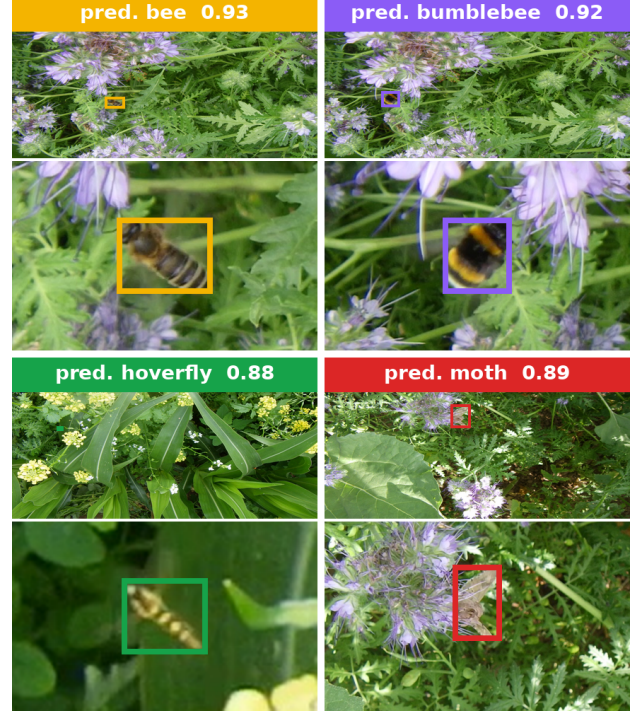


Figure 6. **FinalTest predictions.**

lows iteration-wise cosine decay, with linear warm-up over the first 250 iterations in stage 1 and 100 in stages 2 and 3. The query head weights its quality-focal classification [15], L1 box, and generalized IoU [16] terms by 1, 5, and 2. In stages 1 and 3, we apply horizontal flipping with probability 0.5 and Co-DINO’s 2-branch multi-scale resize/crop augmentation. In stage 2, we replace this pipeline with Mosaic at probability 0.85, photometric distortion, the same horizontal flipping, and multi-scale resizing. At inference, we resize each keyframe, preserving its aspect ratio, to fit within 2048×1280 , and run one full-frame pass.

3.3. Detector-family comparison

Fig. 2 reports the recorded held-out mAP for plain Co-DINO and 9 configurations spanning TPH-YOLOv5 [17], YOLOv8 [18], RTMDet [19], YOLO11 [20], and YOLOv12 [21]. Because the configurations use different protocols, we use the comparison only for detector selection.

For plain Co-DINO in Fig. 2, class $AP@[.5:.95]$ is [0.616, 0.714, 0.328, 0.592] for bee, bumblebee, hoverfly, and moth: hoverfly trails every other class and carries the highest small-box share in the annotations. This weakness

motivates stage 2’s emphasis on rare and small objects.

3.4. FinalTest result

Our final checkpoint reaches 0.5062 mAP@[.5:.95] on Final-Test, 0.0197 mAP (4.1% relative) above the second entry. Fig. 6 shows selected FinalTest outputs.

4. Conclusion

We address the challenge’s 2 difficulties with a single detector. To handle the few-pixel insects, we select Co-DINO with Swin-L from 10 configurations; to handle the bee-dominated labels, we fine-tune on class-aware crop mosaics and add fixed-ETF supervision. For practical use, we avoid ensembling and test-time augmentation, and take rank 1 on Final-Test at 0.5062 mAP@[.5:.95].

Additional explorations. We also test class-balanced sampling, object-crop copy-paste fine-tuning [22], detector pseudo-labels [23], and synthetic image augmentation adapted from prior work [24]. All underperform the baseline, so we exclude them from the final recipe.

Acknowledgment. This work is supported by IITP grants (No. RS2020-II201336 Artificial intelligence graduate school program (UNIST) 34%; No. RS-2022-II220264 Comprehensive video understanding and generation with knowledge-based deep logic neural network 33%, No. RS-2025-25442149 LG AI STAR Talent Development Program for Leading Large-Scale Generative AI Models in the Physical AI Domain 33%), funded by the Korean government (MSIT).

References

- [1] CVPPA@ECCV. BuzzSpot challenge, 2026.
- [2] Zhuofan Zong, Guanglu Song, and Yu Liu. DETRs with collaborative hybrid assignments training. In *ICCV*, 2023.
- [3] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.
- [4] Junsu Kim, Sumin Hong, Chanwoo Kim, Jihyeon Kim, Yihalem Yimolal Tiruneh, Jeongwan On, Jihyun Song, Sunhwa Choi, and Seungryul Baek. Class-wise buffer management for incremental object detection: An effective buffer training strategy. In *ICASSP*, 2024.
- [5] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. YOLOv4: Optimal speed and accuracy of object detection. *arXiv*, 2020.
- [6] Yibo Yang, Shixiang Chen, Xiangtai Li, Liang Xie, Zhouchen Lin, and Dacheng Tao. Inducing neural collapse in imbalanced learning: Do we really need a learnable classifier at the end of deep neural network? In *NeurIPS*, 2022.
- [7] Yibo Yang, Haobo Yuan, Xiangtai Li, Zhouchen Lin, Philip H. S. Torr, and Dacheng Tao. Neural collapse inspired feature-classifier alignment for few-shot class-incremental learning. In *ICLR*, 2023.
- [8] Junsu Kim, Yunhoe Ku, Dongyoon Han, and Seungryul Baek. Beyond synthetic replays: Turning diffusion features into few-shot class-incremental learning knowledge. *arXiv*, 2025.
- [9] Shuai Shao, Zeming Li, Tianyuan Zhang, Chao Peng, Gang Yu, Xiangyu Zhang, Jing Li, and Jian Sun. Objects365: A large-scale, high-quality dataset for object detection. In *ICCV*, 2019.
- [10] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *ECCV*, 2014.
- [11] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In *ICLR*, 2023.
- [12] Vardan Papyan, X. Y. Han, and David L. Donoho. Prevalence of neural collapse during the terminal phase of deep learning training. *PNAS*, 2020.
- [13] COCO API. Microsoft COCO Evaluation Software, 2015.
- [14] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. PyTorch Distributed: Experiences on accelerating data parallel training. *PVLDB*, 2020.
- [15] Xiang Li, Wenhui Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *NeurIPS*, 2020.
- [16] Hamid Rezatoofghi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *CVPR*, 2019.
- [17] Xingkui Zhu, Shuchang Lyu, Xu Wang, and Qi Zhao. TPH-YOLOv5: Improved YOLOv5 based on transformer prediction head for object detection on drone-captured scenarios. In *ICCVW*, 2021.
- [18] Ultralytics. YOLOv8 software, 2023.
- [19] Chengqi Lyu, Wenwei Zhang, Haian Huang, Yue Zhou, Yudong Wang, Yanyi Liu, Shilong Zhang, and Kai Chen. RTMDet: An empirical study of designing real-time object detectors. *arXiv*, 2022.
- [20] Ultralytics. YOLO11 software, 2024.
- [21] Yunjie Tian, Qixiang Ye, and David Doermann. YOLOv12: Attention-centric real-time object detectors. In *NeurIPS*, 2025.
- [22] Golnaz Ghiasi, Yin Cui, Aravind Srinivas, Rui Qian, Tsung-Yi Lin, Ekin D. Cubuk, Quoc V. Le, and Barret Zoph. Simple copy-paste is a strong data augmentation method for instance segmentation. In *CVPR*, 2021.
- [23] Junsu Kim, Yunhoe Ku, Jihyeon Kim, Junuk Cha, and Seungryul Baek. VLM-PL: Advanced pseudo labeling approach for class incremental object detection via vision-language model. In *CVPRW*, 2024.
- [24] Junsu Kim, Yunhoe Ku, and Seungryul Baek. Can synthetic images conquer forgetting? Beyond unexplored doubts in few-shot class-incremental learning. In *ICCVW*, 2025.